

RAMSES Data Structure and the LaRT Interface

Contents

1. Overview of RAMSES	1
2. Output Directory Layout	2
3. The Info File (<code>info_NNNNN.txt</code>)	2
4. The AMR File (<code>amr_NNNNN.outCCCCC</code>)	3
4.1 Header Records	3
4.2 Level Blocks	4
4.3 Oct Structure and Leaf Identification	4
4.4 Cell Position Calculation	4
5. The Hydro File (<code>hydro_NNNNN.outCCCCC</code>)	5
5.1 Header Records	5
5.2 Variable Ordering	5
6. Physical Unit Conversions	6
6.1 Hydrogen Number Density	6
6.2 Velocity	6
6.3 Temperature	6
7. The LaRT Reader: Step-by-Step	7
7.1 Domain Skipping	7
8. File Naming Convention	8
9. LaRT Input for RAMSES Data	8
10. Generic Text Format vs. RAMSES Binary	9
11. Known Limitations and Assumptions	9

1. Overview of RAMSES

RAMSES [1] is an adaptive mesh refinement (AMR) code for astrophysical hydrodynamics and N-body dynamics. It decomposes the computational domain into an octree hierarchy of grids, refining regions of interest to arbitrarily high resolution. The MPI decomposition uses a space-filling curve (Peano–Hilbert ordering) to distribute octs among CPU ranks.

LaRT v2.00 can read RAMSES binary output directly via `amr_type = 'ramses'`. This document describes the RAMSES binary format in detail and explains how `read_ramses_amr.f90` translates it into the flat leaf-cell arrays required by the LaRT octree engine.

2. Output Directory Layout

A RAMSES snapshot is stored in a directory named `output_NNNNN/` where `NNNNN` is the five-digit output number. For a run with N_{cpu} MPI tasks the snapshot contains the following per-CPU files:

```
1 output_00010/
2   info_00010.txt           ! global metadata: units, cosmology, etc.
3   amr_00010.out00001      ! AMR tree for CPU 1
4   amr_00010.out00002      !   ...
5   ...
6   amr_00010.outNNNNN      ! AMR tree for CPU N
7   hydro_00010.out00001    ! hydrodynamic variables for CPU 1
8   hydro_00010.out00002
9   ...
10  hydro_00010.outNNNNN
11  gravity_00010.out00001   ! gravitational potential (optional)
12  part_00010.out00001     ! particle data (optional)
```

All `.out*` files are Fortran *unformatted sequential* binary files (each record bracketed by 4-byte integer record markers on little-endian systems).

3. The Info File (`info_NNNNN.txt`)

`info_NNNNN.txt` is a plain-text file containing global simulation parameters. The fields relevant to LaRT are listed below.

Key	Type	Meaning
<code>ncpu</code>	integer	Number of MPI ranks used for this snapshot
<code>ndim</code>	integer	Number of spatial dimensions (always 3 for 3-D runs)
<code>nlevelmax</code>	integer	Maximum AMR refinement level used
<code>boxlen</code>	real	Box length in <i>code units</i> (see below)
<code>unit_l</code>	real	Length unit: 1 code length = <code>unit_l</code> cm
<code>unit_d</code>	real	Density unit: 1 code density = <code>unit_d</code> g cm ⁻³
<code>unit_t</code>	real	Time unit: 1 code time = <code>unit_t</code> s
<code>gamma</code>	real	Adiabatic index γ of the EOS

The velocity unit is derived as `unit_v = unit_l/unit_t` (cm s⁻¹).

Cosmological runs. For cosmological simulations, `boxlen` is in Mpc/h and `unit_l` incorporates the scale factor a so that $L_{\text{phys}} = \text{boxlen} \times \text{unit}_l$ cm is the comoving box size. LaRT reads the physical box size as $L_{\text{cm}} = \text{boxlen} \times \text{unit}_l$.

Example info file (non-cosmological).

```

1  ncpu      =      64
2  ndim      =       3
3  nlevelmax =      12
4  ngridmax  =    500000
5  boxlen    =  1.0000000000000000E+02
6  unit_l    =  3.0856775814913670E+21    ! 1 kpc in cm
7  unit_d    =  6.7707722919716770E-25
8  unit_t    =  3.0856775814913670E+14
9  gamma     =  1.6666666666666667E+00

```

In this example the box is 100 kpc on a side.

4. The AMR File (`amr_NNNNN.outCCCCC`)

Each CPU file stores the AMR tree data for that CPU's domain. The file consists of a *header section* followed by a sequence of *level blocks*, one per AMR level.

4.1 Header Records

The AMR header records are read in the following order:

Record	Variable(s)	Description
1	<code>ncpu</code>	Total number of MPI ranks
2	<code>ndim</code>	Number of dimensions
3	<code>nx, ny, nz</code>	Coarse grid size (typically $1 \times 1 \times 1$ or $2 \times 2 \times 2$)
4	<code>nlevelmax</code>	Maximum AMR level
5	<code>ngridmax</code>	Maximum number of grids (octs) per CPU
6	<code>nboundary</code>	Number of boundary domains
7	<code>ngrid_current</code>	Current number of active grids
8	<code>boxlen</code>	Box length (code units)
9–21	(various)	Cosmological parameters, time, etc. (skipped by LaRT)
22	<code>ngridlevel(:, :)</code>	Grid count array: (<code>ncpu</code> , <code>nlevelmax</code>)
23	(level sums)	Skipped
24–27	(boundary grids)	Only present if <code>nboundary > 0</code> (skipped)
28–32	(ordering arrays)	Peano–Hilbert keys, son maps, etc. (skipped)

The ngridlevel array. `ngridlevel(j, l)` gives the number of octs belonging to CPU j at AMR level l . LaRT constructs a combined table `ngridfile(j, l)` that includes both domain and boundary grids.

4.2 Level Blocks

After the header, the file contains one block per AMR level $l = 1, \dots, \text{nlevelmax}$. Within each level, data are stored *domain by domain*: first all domains $j = 1, \dots, \text{ncpu}$, then all boundary domains $j = \text{ncpu} + 1, \dots, \text{ncpu} + \text{nboundary}$.

For each domain j at level l (if `ngridfile(j, l) > 0`):

Record(s)	Variable	Description
1	<code>ind_grid(:)</code>	Global grid indices
2	<code>next(:)</code>	Linked-list next pointer
3	<code>prev(:)</code>	Linked-list previous pointer
4–6	<code>xg(:, 1:3)</code>	Oct center coordinates in code units
7	<code>father(:)</code>	Parent grid index
8–13	<code>nbor(:, 1:6)</code>	Neighbor grid indices (6 faces)
14–21	<code>son(:, 1:8)</code>	Child grid indices; = 0 if a child is a leaf
22–29	<code>cpu_map(:, 1:8)</code>	CPU ownership per child
30–37	<code>ref_map(:, 1:8)</code>	Refinement flags per child

Record counts assume `ndim=3`, `twotondim=8`.

4.3 Oct Structure and Leaf Identification

RAMSES groups cells into *octs*: cubes of $2^3 = 8$ cells sharing a common parent grid cell. Each oct is identified by its *grid index*. Within an oct, the eight child cells are indexed `ind = 1, \dots, 8` with the following ordering:

$$i_x = (\text{ind} - 1) \bmod 2, \quad i_y = \lfloor (\text{ind} - 1) / 2 \rfloor \bmod 2, \quad i_z = \lfloor (\text{ind} - 1) / 4 \rfloor, \quad (1)$$

so that `ind=1` is the $(-x, -y, -z)$ corner and `ind=8` is the $(+x, +y, +z)$ corner.

A child cell at index `ind` in grid g at level l is a **leaf** if and only if

$$\text{son}(g, \text{ind}) = 0. \quad (2)$$

Otherwise the value of `son` is the grid index of the finer oct that occupies that child cell.

4.4 Cell Position Calculation

The center of the coarse grid of the root domain is the origin. For a non-cosmological run with `nx=ny=nz=1`:

$$\mathbf{x}_{\text{bound}} = \left(\frac{n_x}{2}, \frac{n_y}{2}, \frac{n_z}{2} \right). \quad (3)$$

The cell size at level l is $\Delta x_l = 2^{-l}$ (in units where `boxlen` = 1). The offset of child cell `ind` from the oct center is

$$\delta_{\text{ind}} = \left(i_x - \frac{1}{2}, i_y - \frac{1}{2}, i_z - \frac{1}{2}\right) \times \Delta x_l. \quad (4)$$

The absolute position of child `ind` in grid `g` in fractional box units $[0, 1]$ is

$$\tilde{x} = \frac{x_g + \delta_{\text{ind},x} - x_{\text{bound},x}}{n_x} + \frac{1}{2}, \quad (5)$$

and similarly for $\tilde{y}$ and $\tilde{z}$ (the $+1/2$ shifts the origin from the grid corner to the box corner). In the code:

```
xleaf = (xg(j,1) + xc(ind,1) - xbound(1)) / dble(nx) + 0.5d0
```

After reading, leaf positions are converted to physical units (cm):

$$x_{\text{phys}} = \tilde{x} \times L_{\text{box}}, \quad L_{\text{box}} = \text{boxlen} \times \text{unit}_l. \quad (6)$$

5. The Hydro File (`hydro_NNNNN.outCCCCC`)

The hydro file stores the conserved hydrodynamic variables for every cell (leaf and internal alike). It is organized in strict lockstep with the AMR file: one block per level per domain.

5.1 Header Records

Record	Variable	Description
1	<code>ncpu</code>	Number of MPI ranks
2	<code>nvar</code>	Number of hydro variables per cell
3	<code>ndim</code>	Number of dimensions
4	<code>nlevelmax</code>	Maximum AMR level
5	<code>nboundary</code>	Number of boundary domains
6	<code>gamma</code>	Adiabatic index

5.2 Variable Ordering

Standard RAMSES hydro stores the following variables in order:

Index	Variable	Physical meaning
1	ρ	Gas mass density [code units]
2	ρv_x	x -momentum density [code units]
3	ρv_y	y -momentum density [code units]
4	ρv_z	z -momentum density [code units]
5	E	Total energy density (kinetic + thermal) [code units]
6	$Z/Z_\odot$	Metallicity (if <code>metal=.true.</code>)
7+	passive scalars	Additional tracers (simulation-dependent)

Level block structure. For each level l and domain j , the hydro file contains:

1. Two header records (level number and domain number) — skipped.
2. For each cell index $\text{ind}=1,\dots,8$ and each variable $\text{ivar}=1,\dots,\text{nvar}$: one record of length $\text{ngridfile}(j,l)$ containing the values for all grids of domain j at level l .

All records have the same element count $\text{ngridfile}(j,l)$, so the loop structure is:

```

1 do ind = 1, twotondim      ! 8 cells per oct
2   do ivar = 1, nvar        ! variables
3     read(iu_hyd) var(:, ind, ivar) ! (ngrida) values
4   end do
5 end do
    
```

Precision. RAMSES writes hydro variables in double precision (`real*8`) by default. Some configurations compile with single precision (`real*4`). The LaRT reader handles both via the flag `ramses_hydro_prec` (set to 8 or 4 in `read_ramses_amr.f90`).

6. Physical Unit Conversions

6.1 Hydrogen Number Density

The mass density in code units is converted to CGS and then to hydrogen number density assuming a fully ionised gas with mean molecular weight $\mu_H \approx 1$ (one proton mass per hydrogen nucleus):

$$n_H [\text{cm}^{-3}] = \frac{\rho_{\text{code}} \times \text{unit}_d}{m_H}, \quad m_H = 1.6726 \times 10^{-24} \text{ g}. \quad (7)$$

The neutral fraction n_{HI}/n_H is computed separately in `grid_mod_amr.f90` using collisional ionisation equilibrium (CIE) if `par%use_cie_condition = .true.`, otherwise all hydrogen is assumed neutral.

6.2 Velocity

RAMSES stores momentum density ρv_α (variable indices 2–4). The velocity in CGS is:

$$v_\alpha [\text{cm s}^{-1}] = \frac{(\rho v_\alpha)_{\text{code}}}{\rho_{\text{code}}} \times \text{unit}_v, \quad \text{unit}_v = \frac{\text{unit}_l}{\text{unit}_t}. \quad (8)$$

LaRT receives velocities in km s^{-1} (divided by 10^5).

6.3 Temperature

RAMSES variable 5 is the *total energy density* E (kinetic plus thermal). The thermal energy density is:

$$e_{\text{th}} = E - \frac{1}{2} \rho v^2 = E - \frac{(\rho v_x)^2 + (\rho v_y)^2 + (\rho v_z)^2}{2\rho}. \quad (9)$$

The specific thermal energy (per unit mass) in physical units is:

$$\varepsilon [\text{erg g}^{-1}] = \frac{e_{\text{th}}}{\rho_{\text{code}}} \times \text{unit_v}^2. \quad (10)$$

The temperature follows from the ideal gas law with adiabatic index γ and mean molecular weight $\bar{\mu}$ (LaRT assumes $\bar{\mu} = 1.22$ for neutral gas):

$$T [\text{K}] = (\gamma - 1) \varepsilon \frac{\bar{\mu} m_H}{k_B}, \quad k_B = 1.381 \times 10^{-16} \text{ erg K}^{-1}. \quad (11)$$

A floor of $T = 10 \text{ K}$ is applied after conversion.

7. The LaRT Reader: Step-by-Step

The reader in `read_ramses_amr.f90` proceeds as follows:

1. **Read info file.** Extract `ncpu`, `unit_l`, `unit_d`, `unit_t`, `boxlen`, and γ .
2. **Count leaf cells (first pass).** `ramses_count_leaves` opens every `amr_*.outCCCCC` file, reads the `son` arrays for all levels, and counts cells where `son = 0`. This gives the total N_{leaf} needed to pre-allocate output arrays.
3. **Read leaf data (second pass).** `ramses_read_all_cpus` opens *both* the AMR and hydro files simultaneously for each CPU:
 - Reads AMR grids level by level, extracting oct centers `xg` and the `son` array.
 - Reads hydro variables for the same grids.
 - For each child cell with `son = 0` (leaf): computes its physical position, n_H , T , and $\mathbf{v}$, and appends them to the output arrays.
4. **Convert positions.** After the second pass, leaf positions (stored as fractions of `boxlen`) are multiplied by $L_{\text{cm}} = \text{boxlen} \times \text{unit}_l$ to give physical positions in cm. These are then passed to `amr_build_tree` as-is (with `distance2cm = 1`).
5. **Build octree and normalize.** Handled by `grid_mod_amr.f90` (see the AMR description document).

7.1 Domain Skipping

The AMR and hydro files interleave data from all N_{cpu} domains and boundary domains at each level. The LaRT reader only *stores* data for the domain that matches `icpu` (the loop variable); data from other domains are skipped with `skip_records`. This reproduces the original RAMSES read pattern used in post-processing tools such as PYMSES and YT.

8. File Naming Convention

File pattern	Content
output_NNNNN/info_NNNNN.txt	Global metadata
output_NNNNN/amr_NNNNN.outCCCC	AMR tree for CPU CCCCC
output_NNNNN/hydro_NNNNN.outCCCC	Hydro variables for CPU CCCCC
output_NNNNN/gravity_NNNNN.outCCCC	Gravitational potential (optional)
output_NNNNN/part_NNNNN.outCCCC	Particle data (optional)

where NNNNN is the zero-padded 5-digit snapshot index and CCCCC is the zero-padded 5-digit CPU rank (1-based).

9. LaRT Input for RAMSES Data

```

1 &parameters
2   par%use_amr_grid = .true.
3   par%amr_type     = 'ramses'
4   par%amr_file     = '/path/to/simulation' ! repository directory
5   par%amr_snapnum  = 10                  ! snapshot number (NNNNN)
6   par%no_photons   = 1e6
7   par%taumax       = 1.0e7
8   par%DGR          = 0.0
9   par%spectral_type = 'voigt'
10  par%source_geometry = 'point'
11  par%xs_point      = <x_cm> ! source position in cm
12  par%ys_point      = <y_cm>
13  par%zs_point      = <z_cm>
14  par%nxfreq        = 121
15  par%nxim          = 64
16  par%nyim          = 64
17 /

```

Notes.

- `par%amr_file` is the *repository directory* containing `output_NNNNN/` (not the output directory itself).
- `par%amr_snapnum` selects the snapshot (the integer NNNNN).
- For RAMSES data, `par%distance2cm` is automatically set to 1.0 because the reader returns positions in cm (pre-multiplied by `unit_1`). The `par%distance_unit` parameter is ignored.
- `par%taumax` is specified as the line-center optical depth from the box center to the $+z$ boundary.

- Source position (`xs_point` etc.) must be given in cm, matching the units of the RAMSES leaf positions after conversion.
- If `par%use_cie_condition = .true.`, the neutral fraction n_{HI}/n_H is computed from the temperature via collisional ionisation equilibrium; otherwise all hydrogen is assumed neutral.

10. Generic Text Format vs. RAMSES Binary

For cases where RAMSES binary output is unavailable (e.g., data from other simulation codes, or hand-constructed test grids), LaRT provides the generic text format described separately. The two formats differ in the following ways:

Property	RAMSES binary	Generic text
File format	Fortran unformatted binary	Plain text
Position unit	cm (auto, from <code>unit_1</code>)	Code units (set by <code>distance_unit</code>)
<code>par%amr_type</code>	<code>'ramses'</code>	<code>'generic'</code>
<code>par%amr_snapnum</code>	required	not used
Multi-CPU support	yes (reads all <code>.outCCCCC</code> files)	no (single file)
Physical units	full CGS conversion built in	user's responsibility

The generic text file format is:

```

1 nleaf boxlen
2 x y z level nH[cm^-3] T[K] vx[km/s] vy[km/s] vz[km/s]
3 ...

```

The Python utility `examples/python/make_amr_grid.py` provides the `AMRGrid` class to construct and write these files from arbitrary density, temperature, and velocity fields.

11. Known Limitations and Assumptions

1. **Neutral fraction.** The RAMSES reader sets n_H (total hydrogen density, not n_{HI}). The neutral fraction is computed later in `grid_create_amr` using either CIE (if `par%use_cie_condition = .true.`) or the assumption of full neutrality. For simulations with radiation feedback or on-the-fly ionisation, a custom neutral fraction field from a passive scalar may be needed.
2. **Mean molecular weight.** The temperature calculation in `ramses_read_all_cpus` assumes $\bar{\mu} = 1.22$ (fully neutral primordial gas). For ionised or metal-enriched gas this gives an overestimate.
3. **Hydro precision.** The variable `ramses_hydro_prec` (default 8) must be changed to 4 for simulations compiled with single-precision hydro.

4. **Non-standard variable ordering.** If the RAMSES simulation includes passive scalars before the energy variable (e.g., some MHD setups), the variable index for energy ($E = \text{var } 5$) may be wrong. Check the RAMSES namelist (`hydro.nml`) for the actual ordering.
5. **Memory.** All leaf cells are broadcast to every MPI rank in LaRT. For large RAMSES outputs ($\gtrsim 10^7$ leaf cells) this may require significant RAM per node.

References

- [1] Teyssier, R. 2002, A&A, 385, 337 (*RAMSES: a new high-resolution n-body and hydrodynamics code for cosmological simulations*)